

Building an AI Agent: Payment Fraud Transaction Triage

Author: Gunasekaran Pasupathy

Date: 6/28/2026

Github: <https://github.com/gunasekaran-pasupathy-research/Semester-1-MSAAI-501-AIML/tree/main/notebooks/Module%201>

This notebook builds an AI agent using LlamaIndex and Google Gemini. I am only using these two tools, based on the professor's Slack clarification. In this design, LlamaIndex is not just passing the question to Gemini. It is also used as a retrieval/memory layer, where fraud policy rules are indexed and searched before the agent makes a decision.

The agent is designed as a payment fraud triage agent. In simple terms, it looks at a transaction, checks risk signals, searches fraud policy, reviews customer history, and then decides whether the transaction should be approved, held for review, or declined.

In this notebook:

- Gemini is used as the reasoning engine.
- LlamaIndex is used to retrieve fraud policy from a small knowledge base.
- ReActAgent is used so the agent can reason, choose tools, observe results, and take action.
- The agent has 4 sensors: transaction details, risk signals, fraud-policy retrieval, and customer history.
- The agent has 4 actuators: submit a triage decision, open a fraud case, notify the cardholder, and update the risk blocklist.

Note: Since this is only a lab prototype, I use mock data instead of connecting to a real payment system. The mock data represents the transaction store, risk service, customer database, and downstream fraud systems. This keeps the example simple, safe, and easy to understand while still showing how an AI agent could work in a real payment fraud scenario.

```
In [5]: # Core LlamaIndex + Gemini integration packages (python-dotenv loads the API
%pip install -q llama-index llama-index-llms-google-genai llama-index-embedc
```

Note: you may need to restart the kernel to use updated packages.

1. Connect to the Gemini API

In this step, I connect the notebook to Google Gemini. Gemini is used as the main language model for reasoning, and the Gemini embedding model is used by LlamaIndex to create searchable representations of the fraud policy documents.

I set both of these globally using Settings, so the rest of the notebook can use the same Gemini model and embedding model when building the index and running the agent.

```
In [12]: import os
from llama_index.core import Settings
from llama_index.llms.google_genai import GoogleGenAI
from llama_index.embeddings.google_genai import GoogleGenAIEmbedding

try:
    from dotenv import load_dotenv, find_dotenv
    load_dotenv(find_dotenv())
except ImportError:
    pass

GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY")

# Setting the model globally
Settings.llm = GoogleGenAI(model="gemini-2.5-flash", api_key=GOOGLE_API_KEY)
Settings.embed_model = GoogleGenAIEmbedding(model_name="gemini-embedding-001")
```

2. Memory layer: LlamaIndex retrieval over a fraud policy

This step creates the agent's memory layer using LlamaIndex. Instead of asking Gemini to answer only from its general training knowledge, I give the agent a small fraud-policy knowledge base to search from.

The fraud rules are stored as Document objects and then indexed using VectorStoreIndex. At run time, the agent can search this index and retrieve the most relevant policy rules before making a fraud decision. This makes the answer more grounded because the agent is using specific policy information, not just guessing.

In a real payment system, these documents could come from fraud rules, card-network guidelines, 3-D Secure guidance, PCI-related procedures, or an internal risk playbook. For this assignment, I use a smaller mock policy library so the logic is easier to follow.

DISCLAIMER : Used Grammarly to fine tune my sentences in the below code

```
In [13]: from llama_index.core import Document, VectorStoreIndex

# Minimal mock for a fraud/risk policy library (one Document per rule area)
# Came up with the below based on my current payments industry knowledge
# DISCLAIMER : Used Grammarly to fine tune my sentences
fraud_policy_docs = [
    Document(text=(
```

```

        "VELOCITY RULE. If a single card is used for more than 5 transactions
        "or the cumulative amount exceeds $2,000 in 60 minutes, treat as high risk.
        "High-velocity transactions should be held for review, not auto-approved."
    ),
    Document(text=(
        "HIGH-RISK MCC RULE. Merchant Category Codes for gift cards (5815–5844),
        "(4829), and crypto exchanges are high risk. For these MCCs above $2,000,
        "step-up verification before approval."
    )),
    Document(text=(
        "GEO / AVS RULE. If the transaction IP country differs from the cardholder's
        "the AVS (address) check fails while the amount exceeds $100, flag as high risk
        "cardholder verification."
    )),
    Document(text=(
        "CVV / CNP RULE. Card-not-present transactions with a failed CVV check are high risk
        "regardless of amount."
    )),
    Document(text=(
        "CHARGEBACK HISTORY RULE. If the customer has 2 or more confirmed fraud
        "last 180 days, lower the review threshold: hold any transaction above $100 for review."
    )),
    Document(text=(
        "TRUSTED CUSTOMER RULE. Customers with account tenure over 24 months and no fraud
        "may be auto-approved up to $500 even if a single soft signal (e.g., failed AVS)."
    )),
    Document(text=(
        "ESCALATION & COMPLIANCE RULE. Any decision to decline or block must be justified
        "triggered it and be written to the audit log. When signals conflict, err on the side of caution
        "insufficient, prefer holding for human review over auto-declining."
    )),
]

# Building the retrieval index
fraud_policy_index = VectorStoreIndex.from_documents(fraud_policy_docs)
policy_query_engine = fraud_policy_index.as_query_engine(similarity_top_k=2)

```

3. Sensors: How the Agent Reads Information

Here I define four sensors as Python functions. These sensors help the agent collect the information it needs before making a fraud decision.

- `get_transaction_details` — reads basic transaction details.
- `get_risk_signals` — reads fraud signals like velocity, AVS, CVV, and device reputation.
- `query_fraud_policy` — searches the LlamaIndex fraud policy memory.
- `get_customer_history` — reads the customer's account history.

LlamaIndex uses the function names and docstrings to understand when each sensor should be used.

```

In [8]: from llama_index.core.tools import FunctionTool

# ---- Mock data ----
TRANSACTION_STORE = {
    "TXN-10293": {"transaction_id": "TXN-10293", "customer_id": "CUST-77", "currency": "USD", "merchant": "psn", "mcc": 5815, "channel": "card_not_present", "ip_country": "RO", "card_count": 1},
    "TXN-10294": {"transaction_id": "TXN-10294", "customer_id": "CUST-12", "currency": "USD", "merchant": "crunchyroll", "mcc": 5812, "channel": "card_present", "ip_country": "US", "card_count": 1}
}

RISK_SIGNAL_SERVICE = {
    "TXN-10293": {"txns_last_60min": 7, "amount_last_60min": 3120.0, "avs_result": "pass", "cvv_result": "pass", "device_new": True, "device_reputation": "good"},
    "TXN-10294": {"txns_last_60min": 1, "amount_last_60min": 38.5, "avs_result": "fail", "cvv_result": "pass", "device_new": False, "device_reputation": "good"}
}

CUSTOMER_DB = {
    "CUST-77": {"customer_id": "CUST-77", "tenure_months": 3, "fraud_chargeback": 1, "avg_ticket": 60.0},
    "CUST-12": {"customer_id": "CUST-12", "tenure_months": 48, "fraud_chargeback": 2, "avg_ticket": 42.0}
}

# SENSOR 1: transaction store
def get_transaction_details(transaction_id: str) -> dict:
    return TRANSACTION_STORE.get(transaction_id, {"error": "transaction not found"})

# SENSOR 2: real-time risk-signal service
def get_risk_signals(transaction_id: str) -> dict:
    return RISK_SIGNAL_SERVICE.get(transaction_id, {"error": "no signals found"})

# SENSOR 3: LlamaIndex retrieval over the fraud policy KB
def query_fraud_policy(question: str) -> str:
    return str(policy_query_engine.query(question))

# SENSOR 4: customer account database
def get_customer_history(customer_id: str) -> dict:
    return CUSTOMER_DB.get(customer_id, {"error": "customer not found"})

# Wrap each sensor as a LlamaIndex tool.
sensor_tools = [
    FunctionTool.from_defaults(fn=get_transaction_details),
    FunctionTool.from_defaults(fn=get_risk_signals),
    FunctionTool.from_defaults(
        fn=query_fraud_policy,
        name="query_fraud_policy",
        description="Retrieve relevant fraud/risk policy rules (LlamaIndex retrieval)"
    ),
    FunctionTool.from_defaults(fn=get_customer_history),
]

```

4. Actuators: How the Agent Takes Action

Here I define four actuators as Python functions. These are the actions the agent can take after reviewing the transaction information.

1. `submit_triage_decision`: records whether to approve, hold, or decline the transaction.
2. `open_fraud_case`: Creates a fraud review case for an analyst.
3. `notify_cardholder`: Sends a verification or alert message to the customer.
4. `update_blocklist`: adds risky cards, devices, or IPs to a blocklist.

For this assignment, these actions are mocked using in-memory lists instead of real payment systems.

```
In [15]: from typing import Literal

AUTH_DECISIONS, FRAUD_CASES, OUTBOUND_NOTIFICATIONS, BLOCKLIST, AUDIT_LOG =

# ACTUATOR 1: authorization system
def submit_triage_decision(transaction_id: str,
                           decision: Literal["APPROVE", "HOLD_FOR_REVIEW", "
                           policy_reason: str) -> str:
    record = {"transaction_id": transaction_id, "decision": decision, "reason":
    AUTH_DECISIONS.append(record)
    AUDIT_LOG.append({"action": "decision", **record})
    return f"Recorded {decision} for {transaction_id}."

# ACTUATOR 2: case-management system
def open_fraud_case(transaction_id: str,
                    priority: Literal["LOW", "MEDIUM", "HIGH"],
                    notes: str) -> str:
    case = {"case_id": f"CASE-{len(FRAUD_CASES) + 1:04d}", "transaction_id":
            "priority": priority, "notes": notes}
    FRAUD_CASES.append(case)
    AUDIT_LOG.append({"action": "open_case", **case})
    return f"Opened {case['case_id']} ({priority}) for {transaction_id}."

# ACTUATOR 3: cardholder communications
def notify_cardholder(customer_id: str,
                     channel: Literal["sms", "email", "push"],
                     message: str) -> str:
    note = {"customer_id": customer_id, "channel": channel, "message": message}
    OUTBOUND_NOTIFICATIONS.append(note)
    AUDIT_LOG.append({"action": "notify", **note})
    return f"Sent {channel} verification to {customer_id}."

# ACTUATOR 4: risk blocklist
def update_blocklist(entity_type: Literal["card", "device", "ip"],
                    value: str, reason: str) -> str:
    entry = {"entity_type": entity_type, "value": value, "reason": reason}
    BLOCKLIST.append(entry)
    AUDIT_LOG.append({"action": "blocklist", **entry})
    return f"Added {entity_type}={value} to blocklist."

actuator_tools = [
```

```

FunctionTool.from_defaults(fn=submit_triage_decision),
FunctionTool.from_defaults(fn=open_fraud_case),
FunctionTool.from_defaults(fn=notify_cardholder),
FunctionTool.from_defaults(fn=update_blocklist),
]

```

5. Assemble the ReAct Agent

I created the ReActAgent in this section. This agent follows a simple loop: it thinks, chooses a tool, observes the result, and then decides the next step.

Gemini handles the reasoning, while LlamaIndex provides the tools and memory. The system_prompt tells the agent how it should behave, such as checking transaction details, reading risk signals, retrieving fraud policy, and then deciding whether to approve, hold, or decline.

API note: This notebook uses the newer LlamaIndex agent style with ReActAgent and await agent.run(...).

```

In [10]: from llama_index.core.agent.workflow import ReActAgent

SYSTEM_CONTEXT = (
    "You are a payments fraud-triage agent. For each transaction: (1) read t
    "and risk signals, (2) read the customer's history, (3) retrieve the rel
    "query_fraud_policy, then (4) decide APPROVE, HOLD_FOR_REVIEW, or DECLIN
    "Always ground the decision in a retrieved policy rule and record it wit
    "Open a case for anything held or declined, notify the cardholder when v
    "and only blocklist on strong evidence. When signals conflict or evidenc
    "HOLD_FOR_REVIEW over DECLINE."
)

agent_tools = sensor_tools + actuator_tools

fraud_triage_agent = ReActAgent(
    tools=agent_tools,
    llm=Settings.llm,
    system_prompt=SYSTEM_CONTEXT,
)

```

I was able to run this notebook using a valid GOOGLE_API_KEY.

In this example, the agent triages TXN-10293, which is a high-risk sample transaction. It includes several suspicious signals, such as a high amount, gift-card merchant, card-not-present channel, country mismatch, AVS failure, high velocity, and previous fraud chargebacks.

When executed, the agent uses the sensors to read transaction details, risk signals, policy rules, and customer history. Then it uses the actuators to record a decision and take follow-up actions such as opening a fraud case or notifying the cardholder.

Since the current LlamaIndex agent API is async, the notebook uses: `await agent.run(...)`

```
In [17]: response = await fraud_triage_agent.run("Triage transaction TXN-10293 and ta
print(response)

# A second, low-risk example (should APPROVE):
response = await fraud_triage_agent.run("Triage transaction TXN-10294.")
print(response)

# Print the actuator side effects so we can see
print(AUTH_DECISIONS, FRAUD_CASES, OUTBOUND_NOTIFICATIONS, BLOCKLIST)
```

Transaction TXN-10293 has been triaged as 'HOLD_FOR_REVIEW' due to high velocity (7 transactions and \$3120 in 60 minutes), a high-risk gift card purchase (MCC 5815) for an amount over \$200, a new device with poor reputation, AVS failure, and the customer having a history of fraud chargebacks. A high-priority fraud case (CASE-0002) has been opened for further investigation, and a verification SMS has been sent to the cardholder (CUST-77).

The transaction TXN-10294 has been approved. The decision is based on the policy that allows auto-approval for customers with an account tenure exceeding 24 months and no history of fraud disputes, for transactions up to \$500, provided there are no high velocity signals. Customer CUST-12 has a tenure of 48 months, no recent fraud chargebacks, and the transaction amount of \$38.50 is well within the limit, with no high velocity indicators.

```
[{'transaction_id': 'TXN-10293', 'decision': 'HOLD_FOR_REVIEW', 'reason': 'Transaction involves high-risk gift card MCC (5815) with amount > $200, mandating 3-D Secure step-up. Customer has 2 confirmed fraud chargebacks in 180 days, and transaction amount > $50, requiring manual review.'}, {'transaction_id': 'TXN-10293', 'decision': 'HOLD_FOR_REVIEW', 'reason': 'Transaction meets high velocity criteria (7 transactions and $3120 in 60 minutes) and is a high-risk gift card purchase (MCC 5815) for an amount over $200, requiring 3-D Secure step-up verification.'}, {'transaction_id': 'TXN-10294', 'decision': 'APPROVE', 'reason': 'Customer tenure exceeds 24 months, no fraud history, transaction amount under $500, and no high velocity signals.'}] [{"case_id": "CASE-0001", "transaction_id": "TXN-10293", "priority": "HIGH", "notes": "Transaction at high-risk gift card merchant (MCC 5815) for $845. AVS failed, new device with poor reputation, and high velocity (7 txns, $3120 in 60min). Customer has 2 prior fraud chargebacks in 180 days and average ticket of $60. Policy mandates 3-D Secure step-up and manual review due to amount and customer history."}, {"case_id": "CASE-0002", "transaction_id": "TXN-10293", "priority": "HIGH", "notes": "Transaction TXN-10293 held for review due to high velocity (7 txns, $3120 in 60 min), high-risk gift card purchase ($845 at MCC 5815), new device with poor reputation, AVS failure, and customer CUST-77 having 2 prior fraud chargebacks and a significantly higher than average transaction amount."}] [{"customer_id": "CUST-77", "channel": "sms", "message": "Did you attempt an $845 purchase at GiftCardMart? Please reply YES or NO to verify."}, {"customer_id": "CUST-77", "channel": "sms", "message": "Did you attempt an $845 purchase at GiftCardMart? Please reply YES or NO to verify."}] []
```

Part 2: Technical Goal, Success Metrics, and Justification

The Goal

The goal of this agent is to catch likely fraudulent transactions before approval, while avoiding unnecessary blocks for real customers.

For each transaction, the agent chooses one action:

- APPROVE
- HOLD_FOR_REVIEW
- DECLINE

The agent should reduce fraud loss, but it should not decline everything. A good fraud agent needs to balance fraud prevention, customer experience, review cost, and auditability.

Success Metrics

Success Area	Metric	Weight
Fraud detection	Catch rate for confirmed fraud	30%
Customer experience	False-decline rate	25%
Business impact	Money saved per 1,000 transactions	20%
Compliance	Decision cites policy and is logged	10%
Analyst efficiency	Quality of cases opened	10%
Speed	Time to decision	5%

Why These Weights

I gave the highest weight to fraud detection because the main purpose is to stop fraud. I also gave high weight to false declines because blocking real customers can hurt trust and revenue.

Business impact matters because the agent should help reduce real cost, not just improve a model score. Compliance, analyst efficiency, and speed are smaller weights, but they are still needed for a real payment system.

Justification

I chose this goal because fraud triage is a trade-off problem. The agent has to decide when to approve, hold, or decline a transaction. It should catch risky transactions, but also avoid hurting good customers. Since the agent uses sensors, policy retrieval, and actuators, this goal fits the design of the notebook.

Part 3: Top 2 Additional Functionalities

If I could add two more features, I would choose these:

1. Feedback Loop From Real Outcomes

Right now, the agent uses fixed mock rules and policy documents. In a real system, I would want the agent to learn from what happened later.

For example, it should compare its decision with later outcomes such as:

- confirmed fraud chargebacks
- customer saying "this was me"
- analyst review results

This would help the agent improve over time. If it is holding too many good transactions, the rules can be adjusted. If it is missing fraud, the risk threshold can be made stronger.

I chose this feature because fraud patterns change often, so the agent should not stay static.

2. Cross-Transaction Pattern Detection

The second feature I would add is the ability to look across many transactions, not just one transaction at a time.

For example, the agent could check whether:

- many cards are using the same device
- many accounts are coming from the same IP address
- the same merchant has unusual activity
- one customer suddenly has many failed attempts

This would help detect organized fraud patterns that may not be obvious from a single transaction.

I chose this feature because real fraud is often connected across multiple users, devices, cards, and merchants. A graph-style memory would make the agent much stronger.

Note : I chose these based on what I observed at day-to-day work.